Boolean Compressed Sensing

Delft University of Technology
EE4740 Data Compression: Entropy and Sparsity Perspectives
April 11, 2024

Joris van de Weg
5174821

Burcu Erarslanoglu
5948355

I. Introduction

Group testing is the method of efficiently identifying a relatively small class with as few tests as possible. This is used in many applications such as biology, communications, information theory, and data science [1]. Some examples of these group testing applications are disease diagnosis, genomics, screening libraries of DNA clones, and quality control in manufacturing [2]. Data compression algorithms can also benefit from the group testing principles to store and retrieve data more effectively. The method of group testing was first used during World War II to test enlisted soldiers for Syphilis. Instead of performing individual blood tests for every soldier, a pool of blood samples was created, and the test was carried out simultaneously to detect if at least one soldier in the pool had Syphilis [2].

II. Motivation

When dealing with large datasets; time, resources, and effort are important metrics to consider across a wide range of disciplines. Group testing proves itself to be a valuable technique in scenarios requiring tests or measurements to detect rare characteristics within a dataset. Since group testing reduces the number of tests needed to identify positives significantly, it enhances operational efficiency. This efficiency is important when managing the sparsity of rare events or characteristics and ensures that resources are optimally utilized to achieve the most effective outcomes.

III. Proposed Methodology

A. Explanation of the Dataset

The given dataset contains 1000 samples, where each represents a sparse binary vector $\mathbf{x}$ of size $N = 100$. The vector $\mathbf{x}$ can be used to model the health status of a group of patients or it can be used to model the number of defective items in a group of items. For any i^{th} item in $\mathbf{x}$, the entry x_i is set to 1 if the item is defective, and 0 otherwise. The term defective is generally used for the 1s in $\mathbf{x}$ and the number of the defectives is denoted with k [1]. The vector $\mathbf{x}$ is inherently sparse because only a small number of items in a group are affected by a specific condition for most cases. Of course, the binary values within the vector $\mathbf{x}$, represented by 0s and 1s, are adaptable to various scenarios, indicating the presence or absence of any event.

B. Analysis Model

A binary matrix $\mathbf{A}$ of size $M \times N$ is defined, for the measurements. M denotes the total number of measurements or tests that are accounted for in $\mathbf{A}$, while N denotes the number of items. The matrix $\mathbf{A}$ is generated using a random pooling strategy, where each entry is independently drawn from a Bernoulli distribution with mean p. The probability mass function that defines this distribution is as follows:

$$P(X = x) = p^x(1-p)^{(1-x)}, \quad \text{for } x \in \{0, 1\} \tag{1}$$

Each row of $\mathbf{A}$ corresponds to a single measurement. The $(m, i)^{\text{th}}$ entry of $\mathbf{A}$ is 1 if the i^{th} entry of $\mathbf{x}$, indicated by x_i, participates in the m^{th} measurement. Also, the m^{th} measurement in $\mathbf{A}$ is the OR of the participating entries of $\mathbf{x}$. This means that for at least one defective item in $\mathbf{x}$, the m^{th} measurement will be 1. The set of M binary measurements forms the vector $\mathbf{y}$. Its element y_m is given by the following:

$$\begin{aligned} y_m = A_{m1}x_1 \vee A_{m2}x_2 \vee \ldots \vee A_{mN}x_N \\ \text{for } m = 1, 2, \ldots, M \end{aligned} \tag{2}$$

C. Analysis Procedures

The primary objective of this project is to accurately estimate the vector $\mathbf{x}$ based on the generated binary measurements $\mathbf{y}$ and the measurement matrix $\mathbf{A}$. This will be done by making use of the sparsity of $\mathbf{x}$ to achieve efficient and effective reconstruction.

For the estimation of $\mathbf{x}$, group testing algorithms from the literature and a greedy algorithm for sparse recovery from the lectures are studied and implemented. The algorithms used are described in detail in this section.

Combinatorial Orthogonal Matching Pursuit (COMP)

The first algorithm that can be used in group testing to identify defective items is the Combinatorial Orthogonal Matching Pursuit (COMP) algorithm [1]. The main idea of this algorithm is rather simple. The initial thought is that every item could be defective; therefore, the first estimate $\hat{\mathbf{x}}$ is that every item will be marked as defective. Then, for each measurement

where the outcome y_m is negative, all the items included in this measurement pool A_m will be marked as non-defective. After iterating through all the measurements, the remaining items in $\hat{\mathbf{x}}$ that are not marked as non-defective, will be marked as defective.

This method makes sure that it will not result in false negatives. One drawback is that this method will only be practical and computationally efficient for large N, and especially when the prevalence rate of defectives, the proportion of defective items in the set denoted as α, is low.

Definite Defectives (DD)

Another algorithm that can be used in group testing applications is the Definite Defectives (DD) algorithm [1]. This algorithm is an important one for our problem of group testing because it was the first practical group testing algorithm to provably achieve the optimal rate for Bernoulli designs [1]. The construction of the measurement matrix $\mathbf{A}$ is done according to a Bernoulli design in our problem as well.

In this algorithm, for a negative outcome y_m, any item from that outcome in $\hat{\mathbf{x}}$ is marked as *definitely non-defective*. Therefore, any remaining item in $\hat{\mathbf{x}}$ is considered as *possible defective*. If any *possible defective* item in $\hat{\mathbf{x}}$ is the only *possible defective* item in $\hat{\mathbf{x}}$ from a positive outcome y_m, we call that item in $\hat{\mathbf{x}}$ *definitely defective*. The algorithm outputs the set containing the items that are *definitely defective*.

This algorithm is similar to COMP because, in both of them, some assumptions about the items in the tests are made. In COMP, the core assumption is to assume that every item is defective unless proven otherwise, and in DD the core assumption is to assume that every item is non-defective unless proven otherwise. The assumption made with DD is preferred over the one with COMP under the assumption that defectivity is a rare quality in $\mathbf{x}$, meaning that $\mathbf{x}$ is sparse.

Matching Pursuit (MP)

The last algorithm is the Matching Pursuit (MP) algorithm. Although this algorithm is not found in the literature as a group testing algorithm, it is a greedy algorithm that is used for sparse recovery which is the main aim of the boolean compressed sensing problem. The MP algorithm is designed to recover or approximate a sparse signal vector $\mathbf{x}$ from measurements $\mathbf{y}$, using a matrix $\mathbf{A}$, up to a specified accuracy ε. The algorithm including all its steps is as follows in algorithm 1 below:

Algorithm 1 Matching Pursuit Algorithm [3]

$\mathbf{x} = \mathrm{MP}(\mathbf{A}, \mathbf{y}, \varepsilon)$
$\mathbf{x} \leftarrow \mathbf{0}$
$\mathbf{r} \leftarrow \mathbf{y}$ ▷ $\mathbf{r} = \mathbf{y} - \mathbf{A}\mathbf{x}$
repeat
  $\mathbf{h} \leftarrow \mathbf{A}^T\mathbf{r}$ ▷ Matching step
  $k \leftarrow \underset{1 \le j \le N}{\arg\max} |h_j|$ ▷ Support identification step
  $x_k \leftarrow x_k + \mathbf{a}_k^T\mathbf{r}$ ▷ Update estimate of the sparse vector
  $\mathbf{r} \leftarrow \mathbf{y} - \mathbf{A}\mathbf{x}_k$ ▷ Update residue
until $\|\mathbf{r}\|_2 \le \varepsilon$

The algorithm initializes the parameter which is of interest, $\mathbf{x}$, to zero. The residual $\mathbf{r}$ is then initialized as the difference between $\mathbf{y}$ and the initial approximation of $\mathbf{A}\mathbf{x}$. The iterations then begin, in each of them a **matching step** is carried out to find the atom element (column of $\mathbf{A}$) that has the highest correlation with the current residual $\mathbf{r}$. This is achieved by first defining $\mathbf{h}$ as the transpose of $\mathbf{A}$ multiplied with the residual $\mathbf{r}$ and then selecting the element with the highest absolute correlation. This step is defined as the **support identification step**. The algorithm updates the estimate of $\mathbf{x}$ for each support k, by adjusting the coefficient of the selected atom of $\mathbf{A}$, which is defined as $\mathbf{a}_k$, according to the current residual $\mathbf{r}$. This step is defined as the **update estimate of the sparse vector**. The residual $\mathbf{r}$ is updated each time with the **update residue** step, until the L_2 norm of the residual is less than or equal to ε. With this, it is aimed to make the approximation $\mathbf{x}$ sufficiently close to the true vector.

D. Assessment Metrics

The evaluation of the algorithm's effectiveness involves analyzing the impact of varying the pooling probability p and the number of measurements M on the recovery performance. The assessment is based on two primary metrics: the reconstruction error, measured by the Hamming distance, and the computational complexity involved in the reconstruction.

The Hamming distance between two sequences of equal length is the number of positions at which the corresponding symbols are different. For the boolean compressed sensing problem, Hamming distance will give the number of bit positions in which the two binary strings, $\mathbf{x}$ and its estimate $\hat{\mathbf{x}}$, of the same length differ. This distance can be at most 1 between the i^{th} element of $\mathbf{x}$ and $\hat{\mathbf{x}}$.

For evaluating the performance of the algorithms, the average false positives and false negatives will also be computed. False positives can increase the costs of the measurement process. False negatives, depending on the use case, can give high risks in the case of diseases.

When reconstructing $\hat{\mathbf{x}}$, from $\mathbf{A}$ and $\mathbf{x}$ using the algorithm COMP and DD, the computational complexity is given by $O(nT)$, where n denotes the number of items and T denotes the number of tests conducted as stated in [1]. For the MP algorithm, the computational complexity is given by $O(MN)$ per iteration [3], M and N defining the row and column number of the matrix $\mathbf{A}$, respectively. To assess simulation performances concerning computational complexity, the given big O notations can be used.

IV. Results

In this section, the results achieved using the algorithms COMP, DD, and MP are presented. In fig. 1, fig. 2, and fig. 3, the mean Hamming distance plotted against the number of measurements (M) is shown. In fig. 4, fig. 5, and fig. 6, computing time plotted against M are shown. In all of the figures, M starts at 20. In the legends, the varying pooling probability p for the Bernoulli distribution is given as well.

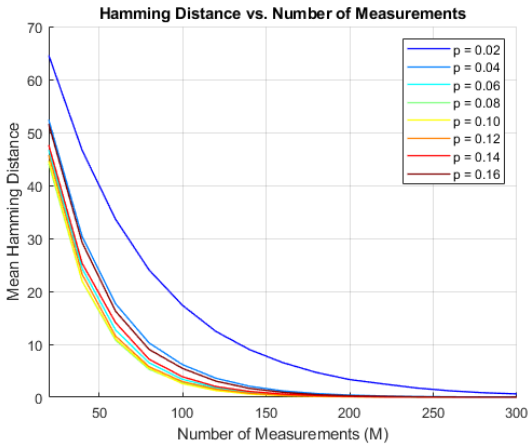

Fig. 1. Mean Hamming distance as a function of measurement number for varying pooling probabilities (p) for the COMP algorithm.

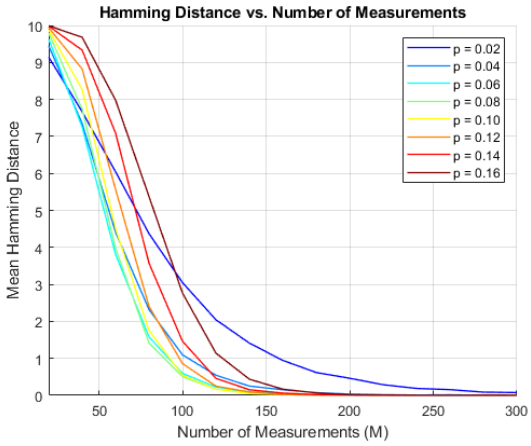

Fig. 2. Mean Hamming distance as a function of measurement number for varying pooling probabilities (p) for the DD algorithm.

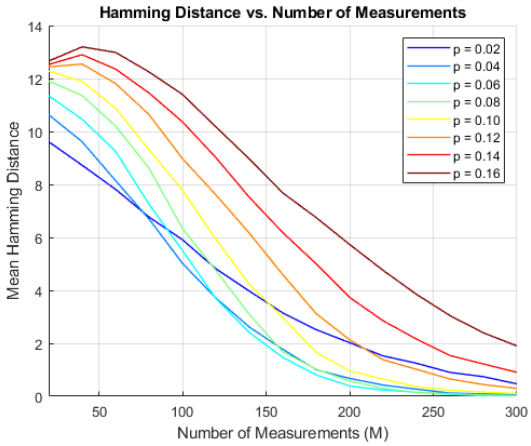

Fig. 3. Mean Hamming distance as a function of measurement number for varying pooling probabilities (p) for the MP algorithm.

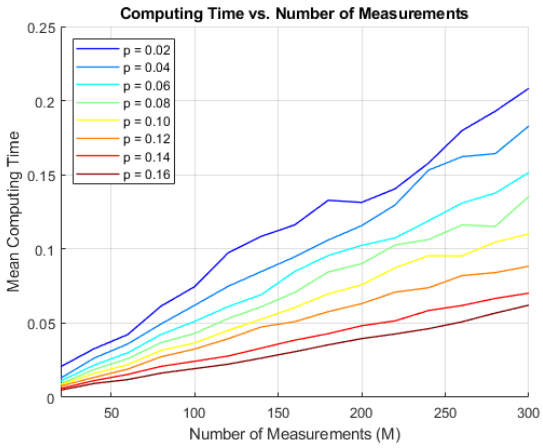

Fig. 4. Mean of the computing time for the algorithm to give an estimate, as a function of measurement number for varying pooling probabilities (p) for the COMP algorithm.

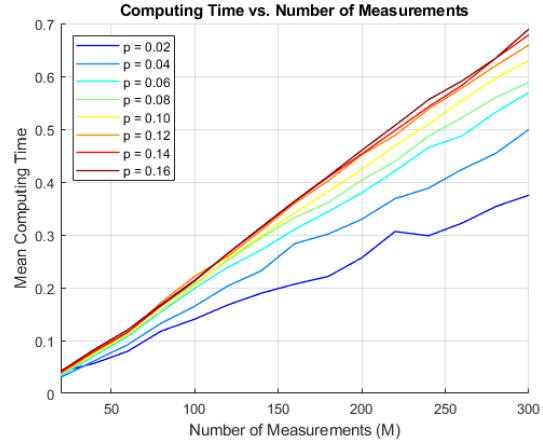

Fig. 5. Mean of the computing time for the algorithm to give an estimate, as a function of measurement number for varying pooling probabilities (p) for the DD algorithm.

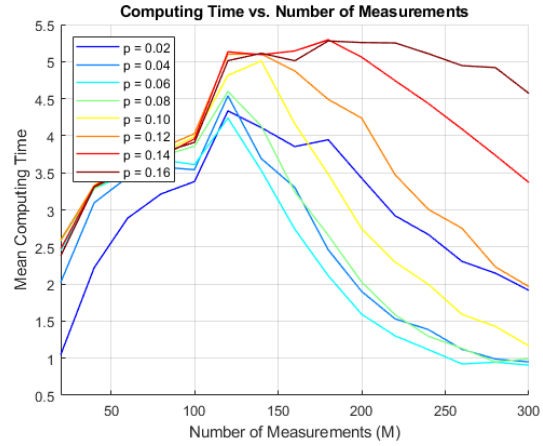

Fig. 6. Mean of the computing time for the algorithm to give an estimate, as a function of measurement number for varying pooling probabilities (p) for the MP algorithm.

V. Discussion

In this section the obtained results at section IV are discussed. First, a comparison for the reconstruction errors and then for the computation times is done.

Looking at the result for the MP algorithm in fig. 3, it can quickly be seen that the algorithm needs more measurements to achieve the true state vector $\mathbf{x}$ when compared with the other two algorithms. For a few of the low pooling probabilities, the algorithm starts to converge when 300 measurements are considered, but for other p this will take a lot longer. The reason for a lower performance is that, MP is a more standard algorithm that is used for sparse vector recovery compared to the other algorithms which are solely intended for group testing applications.

The fig. 1 and fig. 2 show the computed Hamming distances for algorithms COMP and DD. The algorithms estimate the state vector $\mathbf{x}$ correctly before approximately 200 measurements in these figures. At first sight, fig. 1 and fig. 2 look similar; however, the scaling is different due to the nature of the COMP algorithm. Because the COMP algorithm initializes the estimation vector $\hat{\mathbf{x}}$ to all ones, marking each item as defective, in the first estimation. Since $\mathbf{x}$ is sparse, only containing a few ones, this makes the reconstruction error measured by the Hamming distance high at fewer measurements compared to other algorithms. Looking closer shows that the COMP correctly estimates the $\hat{\mathbf{x}}$ around 225 measurements for almost all probabilities. For DD, this number is lower with 175 measurements. The estimation of $\mathbf{x}$ by COMP never shows any false negatives. The DD does never show any false positives in the estimate. This is in line with the theory.

Now, fig. 6, fig. 4, and fig. 2 will be compared for the computing times of the algorithms. The computing times for the COMP and DD algorithms as shown in fig. 4, and fig. 5 respectively are much lower relative to MP shown in fig. 6. Between COMP and DD, the DD algorithm has a bit of a higher computing time. But this goes with a lower number of measurements needed to get the same reconstruction error. The behaviour of the computing times shows a linear trend for COMP and DD.

A higher pooling probability means that the computation time is lower for COMP but higher for DD. So, with more items per test, there will be a higher probability of tests returning positive, because there is a higher chance that a defective is included in the measurement pool. For COMP, fewer negative results mean that there are fewer opportunities to exclude items, which paradoxically simplifies the algorithm's task. For DD the increase in items per test will result in more complex scenarios because DD would then need to iterate over a greater combination of items and tests to resolve these complexities.

The MP algorithm has a different behaviour for its computing time, which can be described by its implementation. When the residual $\mathbf{r}$ is achieved before the maximum amount of iterations, the algorithm returns the estimated value earlier. So it seems with enough measurement, the columns will be better defined. With a better representation and higher chances of selecting highly correlated atoms, the residual tends to decrease more rapidly.

VI. Conclusions

In this report, the problem of boolean compressed sensing within the framework of group testing was explored by using the algorithms COMP, DD, and MP to effectively recover sparse measurements. The simulations conducted focused on the interplay between the number of measurements M, reconstruction accuracy using Hamming distance, and pooling probability p.

The comparative analysis done, revealed that out of the three algorithms, MP performed the worst in both Hamming distance convergence and computing time. Although the computing goes down after a certain amount of measurements, this is most likely because the residual $\mathbf{r}$ is achieved before the maximum amount of iterations, and the algorithm returns the estimated value earlier. Although the other two algorithms show similar results, DD proves to be the better method for achieving the true state vector $\mathbf{x}$ with the lowest amount of measurements.

Apart from these three algorithms, some work was also put into the non-negative LASSO [4]. The implementation of this algorithm took a long time to converge and only achieved good results for a high number of measurements.

As future work, the algorithms that were not considered in this paper, but were in [1] and [5] could be implemented. Different algorithms may be combined to see whether they together achieve better estimates of $\hat{\mathbf{x}}$ for reduced reconstruction errors. In this report, the reconstruction errors were shown using the metric of mean hamming distance. Various other reconstruction error metrics such as mean square error (MSE) can be used to compare the performance of different algorithms. The algorithms can be implemented on different $\mathbf{A}$ matrices that are constructed with higher p values from the Bernoulli distribution, to see the differences and common trends. Also, the algorithms can be adapted to more complex scenarios of group testing where noise is present. This would definitely enhance the algorithms' applicability across a broader spectrum of group testing scenarios.

To conclude, the literature study, implementation of the algorithms in MATLAB, and the results achieved demonstrated the effectiveness and pitfalls of different algorithms in the boolean compressed sensing problem of group testing. The presentation [6], presentation video [7], and MATLAB code [8] for this project can be found in the References.

References

[1] M. Aldridge, O. Johnson, and J. Scarlett, "Group testing: an information theory perspective," *CoRR*, vol. abs/1902.06002, 2019.

[2] G. K. Atia and V. Saligrama, "Boolean compressed sensing and noisy group testing," *CoRR*, vol. abs/0907.1061, 2009.

[3] G. Joseph, N. J. Myers, "Greedy algorithms for sparse recovery (lecture 8) slides." Link, Delft University of Technology, 2024.

[4] S. Ghosh, R. Agarwal, M. A. Rehan, S. Pathak, P. Agrawal, Y. Gupta, S. Consul, N. Gupta, Ritika, R. Goenka, A. Rajwade, and M. Gopalkrishnan, "A compressed sensing approach to pooled rt-pcr testing for covid-19 detection," 2021.

[5] C. L. Chan, S. Jaggi, V. Saligrama, and S. Agnihotri, "Non-adaptive group testing: Explicit bounds and novel algorithms," *CoRR*, vol. abs/1202.0206, 2012.
[6] Burcu Erarslanoglu and Joris van de Weg, "Presentation," 2024. Link.
[7] Burcu Erarslanoglu and Joris van de Weg, "Video," 2024. Link.
[8] Burcu Erarslanoglu and Joris van de Weg, "Code," 2024. Link.